Advanced Rust (2026): Macros

Lecture 3

Lukáš Hozda

Spring 2026

MFF CUNI

Metaprogramming

Metaprogramming

- Program manipulates program structure

Metaprogramming

- Program manipulates program structure
- Transformation may happen before execution or during execution

Metaprogramming

- Program manipulates program structure
- Transformation may happen before execution or during execution
- Macros are one compile-time metaprogramming technique

Metaprogramming

- Program manipulates program structure
- Transformation may happen before execution or during execution
- Macros are one compile-time metaprogramming technique
- Rust uses them heavily in the standard library

Compile-Time vs Runtime

- Compile-time: macros, code generators, templating

Compile-Time vs Runtime

- Compile-time: macros, code generators, templating
- Runtime: reflection, dynamic evaluation, metaclasses

Compile-Time vs Runtime

- Compile-time: macros, code generators, templating
- Runtime: reflection, dynamic evaluation, metaclasses
- Rust is much stronger on the compile-time side

Compile-Time vs Runtime

- Compile-time: macros, code generators, templating
- Runtime: reflection, dynamic evaluation, metaclasses
- Rust is much stronger on the compile-time side
- This is consistent with its zero-cost bias

Kinds of Macros in Rust

- Declarative macros: `macro_rules!` pattern matching over token trees

Kinds of Macros in Rust

- Declarative macros: `macro_rules!` pattern matching over token trees
- Procedural macros: Rust code transforms token streams

Kinds of Macros in Rust

- Declarative macros: `macro_rules!` pattern matching over token trees
- Procedural macros: Rust code transforms token streams
- Attributes are macros too

Kinds of Macros in Rust

- Declarative macros: `macro_rules!` pattern matching over token trees
- Procedural macros: Rust code transforms token streams
- Attributes are macros too
- These two families behave quite differently

Macro Invocation Syntax

- Invocation is always `name!(...)`, `name![...]`, or `name!{...}`

Macro Invocation Syntax

- Invocation is always `name!(...)`, `name![...]`, or `name!{...}`
- Delimiter choice is usually stylistic, not semantic

Macro Invocation Syntax

- Invocation is always `name!(...)`, `name![...]`, or `name!{...}`
- Delimiter choice is usually stylistic, not semantic
- Parsing of the inside is up to the macro matcher or procedural macro code

Standard Library Macros

Macro	Purpose
`assert!`, `assert_eq!`, `assert_ne!`	test assumptions at runtime
`format!`, `println!`, `write!`	format-string frontends
`vec!`	collection construction shorthand
`matches!`	pattern test shorthand
`stringify!`, `concat!`, `compile_error!`	compile-time helpers
`file!`, `line!`, `column!`, `module_path!`	source-location information
`include!`, `include_str!`, `include_bytes!`	embed external file contents